

Exercise 5-3: Get data from a database

Did you use a language model?

A : No, I did not find any use for a language model for this assignment except for writing comments.

```
In [1]: # Imports the Pandas Library
import pandas as pd
```

```
In [2]: # Define the path to the ZIP file containing the RDS-2013-0009.4_SQLITE data.
zip_file_path = 'RDS-2013-0009.4_SQLITE.zip'
```

```
In [3]: # Extract all contents from 'RDS-2013-0009.4_SQLITE.zip' and print each file's n
from zipfile import ZipFile
with ZipFile('RDS-2013-0009.4_SQLITE.zip', mode='r') as zip:
    zip.extractall()
    for file in zip.infolist():
        print(file.filename, file.compress_size, file.file_size)
```

```
Data/FPA_FOD_20170508.sqlite 173776108 795785216
Data/ 0 0
_metadata_RDS-2013-0009.4.xml 11571 51400
Supplements/FPA_FOD_Source_List.pdf 74137 109336
Supplements/ 0 0
_fileindex_RDS-2013-0009.4.html 1098 4398
_metadata_RDS-2013-0009.4.html 13296 89005
```

```
In [ ]:
```

Run queries against the database

```
In [4]: # Import the sqlite3 module to work with SQLite databases.
import sqlite3
# Establish a connection to the SQLite database stored in 'Data/FPA_FOD_20170508
fires_con = sqlite3.connect('Data/FPA_FOD_20170508.sqlite')
# Create a cursor object through the connection to execute SQL commands.
fires_cur = fires_con.cursor()
# Execute an SQL query to list all table names in the database and fetch the res
fires_cur.execute(
    'SELECT name FROM sqlite_master WHERE type="table"').fetchall()
```

```
Out[4]: [('spatial_ref_sys',),
          ('spatialite_history',),
          ('sqlite_sequence',),
          ('geometry_columns',),
          ('spatial_ref_sys_aux',),
          ('views_geometry_columns',),
          ('virts_geometry_columns',),
          ('geometry_columns_statistics',),
          ('views_geometry_columns_statistics',),
          ('virts_geometry_columns_statistics',),
          ('geometry_columns_field_infos',),
          ('views_geometry_columns_field_infos',),
          ('virts_geometry_columns_field_infos',),
          ('geometry_columns_time',),
          ('geometry_columns_auth',),
          ('views_geometry_columns_auth',),
          ('virts_geometry_columns_auth',),
          ('sql_statements_log',),
          ('SpatialIndex',),
          ('ElementaryGeometries',),
          ('KNN',),
          ('Fires',),
          ('idx_Fires_Shape',),
          ('idx_Fires_Shape_node',),
          ('idx_Fires_Shape_rowid',),
          ('idx_Fires_Shape_parent',),
          ('NWCG_UnitIDActive_20170109',)]
```

```
In [5]: # Execute a PRAGMA command to retrieve metadata about the 'Fires' table's column
fires_cur.execute('PRAGMA table_info(Fires)').fetchall()
```

```
Out[5]: [(0, 'OBJECTID', 'INTEGER', 1, None, 1),
(1, 'FOD_ID', 'int32', 0, None, 0),
(2, 'FPA_ID', 'text(100)', 0, None, 0),
(3, 'SOURCE_SYSTEM_TYPE', 'text(255)', 0, None, 0),
(4, 'SOURCE_SYSTEM', 'text(30)', 0, None, 0),
(5, 'NWCG_REPORTING_AGENCY', 'text(255)', 0, None, 0),
(6, 'NWCG_REPORTING_UNIT_ID', 'text(255)', 0, None, 0),
(7, 'NWCG_REPORTING_UNIT_NAME', 'text(255)', 0, None, 0),
(8, 'SOURCE_REPORTING_UNIT', 'text(30)', 0, None, 0),
(9, 'SOURCE_REPORTING_UNIT_NAME', 'text(255)', 0, None, 0),
(10, 'LOCAL_FIRE_REPORT_ID', 'text(255)', 0, None, 0),
(11, 'LOCAL_INCIDENT_ID', 'text(255)', 0, None, 0),
(12, 'FIRE_CODE', 'text(10)', 0, None, 0),
(13, 'FIRE_NAME', 'text(255)', 0, None, 0),
(14, 'ICS_209_INCIDENT_NUMBER', 'text(255)', 0, None, 0),
(15, 'ICS_209_NAME', 'text(255)', 0, None, 0),
(16, 'MTBS_ID', 'text(255)', 0, None, 0),
(17, 'MTBS_FIRE_NAME', 'text(50)', 0, None, 0),
(18, 'COMPLEX_NAME', 'text(255)', 0, None, 0),
(19, 'FIRE_YEAR', 'int16', 0, None, 0),
(20, 'DISCOVERY_DATE', 'realdate', 0, None, 0),
(21, 'DISCOVERY_DOY', 'int32', 0, None, 0),
(22, 'DISCOVERY_TIME', 'text(4)', 0, None, 0),
(23, 'STAT_CAUSE_CODE', 'float64', 0, None, 0),
(24, 'STAT_CAUSE_DESCR', 'text(100)', 0, None, 0),
(25, 'CONT_DATE', 'realdate', 0, None, 0),
(26, 'CONT_DOY', 'int32', 0, None, 0),
(27, 'CONT_TIME', 'text(4)', 0, None, 0),
(28, 'FIRE_SIZE', 'float64', 0, None, 0),
(29, 'FIRE_SIZE_CLASS', 'text(1)', 0, None, 0),
(30, 'LATITUDE', 'float64', 0, None, 0),
(31, 'LONGITUDE', 'float64', 0, None, 0),
(32, 'OWNER_CODE', 'float64', 0, None, 0),
(33, 'OWNER_DESCR', 'text(100)', 0, None, 0),
(34, 'STATE', 'text(255)', 0, None, 0),
(35, 'COUNTY', 'text(255)', 0, None, 0),
(36, 'FIPS_CODE', 'text(255)', 0, None, 0),
(37, 'FIPS_NAME', 'text(255)', 0, None, 0),
(38, 'Shape', 'POINT', 1, None, 0)]
```

Read the results of a SQL query into a DataFrame

```
In [6]: # Execute an SQL command to retrieve metadata about the 'Fires' table structure,
# Use pandas to execute an SQL query that selects specific columns from the 'Fires' table
# This query retrieves state, fire year, discovery date (formatted as datetime),
# The result is stored in a pandas DataFrame named 'fires', allowing for easy data manipulation
fires = pd.read_sql_query(
    '''SELECT STATE, FIRE_YEAR, DATETIME(DISCOVERY_DATE) AS DISCOVERY_DATE,
        FIRE_NAME, FIRE_SIZE, STATE, LATITUDE, LONGITUDE
        FROM Fires''', fires_con)
```

```
In [8]: # Display the first five rows of the 'fires' DataFrame to get an overview of the data
fires.head()
```

Out[8]:

	STATE	FIRE_YEAR	DISCOVERY_DATE	FIRE_NAME	FIRE_SIZE	STATE	LATITUDE	LONGITUDE
0	CA	2005	2005-02-02 00:00:00	FOUNTAIN	0.10	CA	40.036944	-120.036944
1	CA	2004	2004-05-12 00:00:00	PIGEON	0.25	CA	38.933056	-120.036944
2	CA	2004	2004-05-31 00:00:00	SLACK	0.10	CA	38.984167	-120.036944
3	CA	2004	2004-06-28 00:00:00	DEER	0.10	CA	38.559167	-119.036944
4	CA	2004	2004-06-28 00:00:00	STEVENOT	0.10	CA	38.559167	-119.036944

In [7]:

Output a concise summary of the 'fires' DataFrame, including information on each fire.fires.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1880465 entries, 0 to 1880464
Data columns (total 8 columns):
Column Dtype
--- -
0 STATE object
1 FIRE_YEAR int64
2 DISCOVERY_DATE object
3 FIRE_NAME object
4 FIRE_SIZE float64
5 STATE object
6 LATITUDE float64
7 LONGITUDE float64
dtypes: float64(3), int64(1), object(4)
memory usage: 114.8+ MB

In []: